

Engenharia de Software em Ação

Transformando Requisitos de Negócio em Arquitetura Orientada a Objetos

Objetivo e Diretrizes de Implementação: Esta atividade simula demandas reais de desenvolvimento de software em que os requisitos são expressos como *Cards de Tarefa*. Seu papel é projetar a estrutura de classes, garantir o encapsulamento, estabelecer contratos e validar cenários de execução.

- **Pacote de Domínio (model):** Todas as entidades, interfaces e classes de negócio devem ser organizadas neste pacote.
- **Pacote de Execução (main ou app):** As classes de teste contendo o método public static void main(String[] args) devem ser estruturadas neste pacote para validar os comportamentos implementados.

MÓDULO A: O Início do Streaming (Classes, Atributos e Encapsulamento)

Contexto: A empresa está iniciando o desenvolvimento de uma plataforma de streaming e necessita das entidades fundamentais de domínio.

Questão 01 — Entidade Usuário

Crie a classe Usuario no pacote de domínio. Ela deve conter os atributos: nome (String), email (String) e ativo (boolean, representando o status da conta). Para fins didáticos de evolução do código, declare inicialmente todos os atributos com visibilidade pública (public).

Questão 02 — Entidade Vídeo

Crie a classe Video no pacote de domínio. Ela deve possuir os atributos titulo (String) e duracaoMinutos (int).

Questão 03 — Encapsulamento e Regras de Validação

A equipe de testes identificou que o sistema permite cadastrar vídeos com durações negativas ou nulas. Refatore as classes Usuario e Video aplicando o princípio do encapsulamento: modifique a visibilidade de todos os atributos para privada (private) e crie os métodos de acesso (getters e setters). Na classe Video, implemente a seguinte regra de negócio no método de ajuste da duração: se for informado um valor menor ou igual a zero, o sistema deve ignorar o valor inválido e definir a duração como exatamente 1 minuto.

Questão 04 — Construtor com Inicialização Obrigatória

Para assegurar a integridade do banco de dados, nenhum vídeo pode ser instanciado sem dados básicos. Adicione um construtor parametrizado na classe Video que torne obrigatório o fornecimento do titulo e da duracaoMinutos no momento da criação do objeto, garantindo a execução das validações estabelecidas.

Questão 05 — Cenário de Testes e Validação de Estado (Main)

Crie a classe executável MainStreaming no pacote de execução. Instancie um objeto Usuario e dois objetos Video (o primeiro com duração válida de 120 minutos e o segundo tentando atribuir uma duração inválida de -45 minutos). Exiba no console os dados do usuário e a duração final do segundo vídeo para comprovar que a regra de proteção implementada na Questão 03 foi aplicada com sucesso.

MÓDULO B: Identidade, Representação e Memória (toString, Equals, hashCode e Static)

Contexto: Com a evolução da plataforma, torna-se necessário padronizar a exibição de dados e gerenciar a identidade de instâncias em memória.

Questão 06 — Sobrescrita de Representação Textual (toString)

Ao imprimir diretamente uma referência de objeto Usuario, a JVM exibe a representação padrão com identificador de memória. Sobrescreva o método toString() na classe Usuario para retornar uma formatação clara e estruturada no padrão: "Usuário: [Nome] | Contato: [Email] | Ativo: [Status]".

Questão 07 — Comparação de Identidade Lógica (equals)

O sistema atualmente permite cadastros duplicados com o mesmo e-mail. Sobrescreva o método equals(Object obj) na classe Usuario de modo que dois usuários sejam considerados logicamente iguais se, e somente se, possuírem o mesmo endereço de e-mail.

Questão 08 — Constante Global de Classe (static final)

Adicione na classe Usuario uma constante global que armazene o nome oficial da plataforma ("JavaFlix"). Essa informação deve pertencer à classe (não a instâncias isoladas) e não poderá ser modificada em tempo de execução. Utilize os modificadores static e final adequadamente.

Questão 09 — Membros Estáticos e Contador de Instâncias

O setor de métricas necessita contabilizar o total de contas criadas na aplicação. Implemente na classe Usuario um atributo estático privado totalUsuarios que atue como contador global. Incremente este contador a cada execução do construtor e forneça um método estático getTotalUsuarios() para consulta pública da métrica.

Questão 10 — Cenário de Testes e Comparação de Objetos (Main)

Crie a classe executável MainMemoria. Instancie dois objetos Usuario distintos que compartilhem o mesmo endereço de e-mail. Utilize uma estrutura de decisão (if/else) com o método equals() para demonstrar que o sistema reconhece a equivalência lógica entre eles. Em seguida, acesse e imprima o nome da plataforma e a quantidade total de usuários diretamente a partir da classe, sem recorrer às instâncias individuais.

MÓDULO C: O Gateway de Pagamentos (Interfaces, Polimorfismo e Acoplamento Fraco)

Contexto: Implementação do módulo de monetização e assinaturas baseado em contratos operacionais desacoplados de regras condicionais.

Questão 11 — Definição de Contrato Operacional (Interface)

Crie a interface Pagamento no pacote de domínio. Ela deve estabelecer o contrato da operação definindo a assinatura do método void processar(double valor), que deve ser obrigatoriamente implementado por qualquer modalidade de cobrança suportada.

Questão 12 — Implementações Concretas de Pagamento

Crie as classes PagamentoPix e PagamentoCartao, ambas implementando a interface Pagamento. Na classe PagamentoPix, o método processar deve exibir a mensagem: "Gerando QR Code Pix no valor de R\$ [valor]". Na classe PagamentoCartao, o método deve exibir: "Validando limite no cartão para cobrança de R\$ [valor]".

Questão 13 — Inversão de Dependência e Desacoplamento

Crie a classe ProcessadorPagamento contendo o método finalizarCompra(double valor, Pagamento formaPagamento). O método deve receber o valor da transação e a referência da interface Pagamento (evitando o acoplamento com classes concretas), acionando em seguida o método processar do objeto recebido.

Questão 14 — Cenário de Testes e Polimorfismo Dinâmico (Main)

Crie a classe executável MainPagamento. Instancie o ProcessadorPagamento, crie variáveis tipadas com a interface Pagamento instanciando objetos PagamentoPix e PagamentoCartao, e envie ambos para o método finalizarCompra. Demonstre como o polimorfismo permite estender novas formas de pagamento sem necessidade de instruções if/else.

MÓDULO D: Estruturas de Dados e Coleções (Arrays, List e Set)

Contexto: Gestão e controle de acesso aos recursos físicos da sede corporativa através de estruturas de armazenamento especializadas.

Questão 15 — Estrutura Estática com Capacidade Fixa (Array)

Crie a classe Sede no pacote de domínio. Declare dentro dela um atributo privado representando a garagem de veículos da diretoria, estruturado como um array unidimensional (`String[] vagasGaragem`) com dimensão estática e capacidade fixa de exatamente 3 posições.

Questão 16 — Entidade Credencial e Contrato Hash (`hashCode`)

Crie a classe Credencial contendo o atributo `codigoHex` (`String`). Implemente os métodos `equals(Object obj)` e `hashCode()` de forma consistente, garantindo que duas instâncias com o mesmo código alfanumérico gerem o mesmo código hash e sejam tratadas como idênticas em coleções baseadas em hashing.

Questão 17 — Histórico Cronológico com Coleção Dinâmica (List)

Na classe Sede, declare e instancie diretamente na declaração um atributo privado `historicoCatraca` utilizando a coleção dinâmica `List<Credencial>` (com `ArrayList`). Essa estrutura deve permitir o registro cronológico de acessos, aceitando múltiplas entradas da mesma credencial ao longo do dia.

Questão 18 — Acesso Restrito com Coleção de Unicidade (Set)

Na classe Sede, declare e instancie diretamente na declaração um atributo privado `acessoCofre` utilizando a coleção `Set<Credencial>` (com `HashSet`). A estrutura deve assegurar a unicidade dos registros, impedindo a inserção duplicada de uma mesma credencial no ambiente restrito.

Questão 19 — Métodos de Operação e Tratamento de Duplicidade

Na classe Sede, implemente os métodos: `estacionarVeiculo(String placa, int vaga)`, `registrarPassagemCatraca(Credencial c)` e `autorizarEntradaCofre(Credencial c)`. No método do cofre, avalie o retorno booleano do método `add()` do Set: caso a inserção seja recusada devido a duplicidade, exiba a mensagem: *"Alerta de Segurança: Tentativa de acesso duplicado com credencial já ativa no cofre"*.

Questão 20 — Cenário de Testes e Validação de Limites de Estruturas (Main)

Crie a classe executável `MainSeguranca` e realize a sequência de testes:

1. Instancie a Sede e um objeto Credencial.
2. Registre a mesma credencial 2 vezes na Catraca (verificando que ambas as passagens foram registradas no histórico).
3. Tente autorizar a mesma credencial 2 vezes no Cofre (confirmando que a segunda inserção foi bloqueada e disparou o alerta).
4. Preencha as 3 vagas da Garagem e tente adicionar um 4º veículo acessando o índice 3 (`vagasGaragem[3]`), observando o lançamento da exceção `ArrayIndexOutOfBoundsException` para constatar a rigidez dimensional do array primitivo.